# DEV BHOOMI UTTARAKHAND UNIVERSITY

Dehradun, Uttarakhand

(Notified by Govt. of India u/s 2(f) of the U.G.C. Act, 1956)

Object Oriented Programming with C++

2nd Semester | BCA (AI&DS)

ON THE TOPIC OF

## Alien Invasion Game

| | |
|---|---|
| **Submitted by** | Paras Thapa |
| **Course** | BCA (Artificial Intelligence and Data Science) |
| **Semester** | 2nd Semester |
| **Subject** | Object Oriented Programming with C++ |
| **Faculty Name** | Ms. Dhruvi Dagar |
| **College** | Dev Bhoomi Uttarakhand University |
| **Submission Date** | 27 April 2026 |

Academic Year: 2025–2026

# Acknowledgement

First and foremost, I would like to express my sincere gratitude to the Almighty for giving me the strength, knowledge, and determination to complete this project successfully.

I am deeply grateful to my project guide, Ms.Dhruvi Dagar Faculty, Department of Computer Applications, Dev Bhoomi Uttarakhand University, Dehradun, for her invaluable guidance, constant encouragement, and constructive suggestions throughout the course of this project. Her expertise and patience helped me navigate every challenge I encountered during this work.

I would also like to extend my heartfelt thanks to the Head of the Department and all the faculty members of the Department of Computer Applications for their academic support and for creating a conducive learning environment. Their teachings have formed the foundation upon which this project is built.

I express my sincere appreciation to Dev Bhoomi Uttarakhand University for providing excellent infrastructure, laboratory facilities, and academic resources that were essential for the successful completion of this project.

My special thanks go to my classmates and friends who provided motivation, assistance, and healthy discussions during the development phase of this project. Their feedback during the testing phase proved extremely helpful in refining the application.

Last but not least, I am deeply indebted to my parents and family members for their unwavering moral support, encouragement, and belief in me throughout my academic journey. Without their blessings and support, this project would not have been possible.

**Paras Thapa**
BCA (AI&DS) – 2nd Semester
Dev Bhoomi Uttarakhand
University
April 2026

# Chapter 1: Introduction to Topic

The Alien Invasion game is a console-based interactive project developed using the C++ programming language. It is designed to showcase how fundamental programming concepts such as loops, conditional statements, random number generation, and collision detection can be applied to create a real-time gaming experience. In this game, the player controls a spaceship represented by the character 'A', positioned at the bottom of the screen, while aliens represented by 'W' descend from the top. The player can move left or right using keyboard inputs and shoot bullets to destroy the aliens before they reach the ground. The game tracks the player's score, increases difficulty as aliens respawn, and ends when an alien collides with the player's position.

This project highlights the practical application of C++ in building interactive systems and demonstrates how simple logic can be combined with creativity to produce an engaging game. Inspired by classic arcade titles like Space Invaders, the Alien Invasion game emphasizes structured design, responsive controls, and score management, making it both an educational exercise and a fun demonstration of programming skills.

# Chapter 2: Project Objectives

The primary objective of the Alien Invasion game is to design and implement a simple yet interactive shooting game using C++. The project focuses on applying fundamental programming concepts to create a real-time gaming experience. The specific objectives include:

•Developing a user-friendly interface where the player can easily control movement and shooting actions.

•Implementing real-time gameplay mechanics such as player movement, alien descent, bullet firing, and collision detection.

•Designing a scoring system that rewards the player for destroying aliens and tracks progress throughout the game.

•Creating a wave-based enemy system where difficulty increases as aliens respawn faster and in different positions.

•Managing game states effectively, including running, game-over, and restart conditions.

•Demonstrating practical use of programming concepts like loops, functions, arrays, and random number generation in a structured project.

This objective-driven approach ensures that the Alien Invasion game is not only entertaining but also serves as a strong academic exercise, bridging theoretical knowledge with practical application in C++ programming.

# Chapter 3: Scope of alien invasion game

The scope of the Alien Invasion game is centered around designing and implementing a lightweight, console-based shooting game using C++. The project is intended for educational purposes, demonstrating how fundamental programming concepts can be applied to create an interactive application.

Included in Scope:

•A player-controlled spaceship (A) with movement and shooting functionality.

•Alien enemies (W) that spawn at random positions and descend toward the player.

•Collision detection between bullets (|) and aliens, as well as between aliens and the player.

•A scoring system that rewards the player for destroying aliens.

•Game-over conditions when an alien collides with the player's position. Restart and quit options for user convenience.

Excluded from Scope:

•Advanced graphics or 3D rendering (the game uses simple console characters). Multiplayer or online gameplay features.

•Sound effects or background music.

•Save/resume functionality beyond the current session.

# Chapter 4: Software Requirements

Because the Alien Invasion game is implemented in C++ and executed in the terminal, the software requirements are minimal and straightforward. The project does not rely on external frameworks or advanced graphics libraries, making it accessible for students and learners working in a basic development environment.

Required Software:

•**C++ Compiler**: A standard compiler such as MinGW (G++) or Microsoft Visual C++ to compile and run the program.

•**Integrated Development Environment (IDE)**: Any text editor or IDE can be used, such as Visual Studio Code, Code::Blocks, or even a simple editor like Notepad++. Since the game runs in the terminal, IDE choice is flexible.

•**Operating System**: Windows 10/11 is recommended because the program uses <conio.h> and <windows.h> libraries for console handling.

•**Terminal/Console**: The Windows Command Prompt or PowerShell is sufficient to execute the compiled program.

Optional Software:

•Debugging tools within the IDE for testing and troubleshooting.

•GitHub or other version control systems for managing project files.

This lightweight requirement ensures that the game can be developed and executed on any standard personal computer or college lab system without additional installations.

# Chapter 5: Backend – Core Logic

The backend of your Alien Invasion game is where the main gameplay mechanics are implemented. Since you are running it in the terminal using C++, the logic is handled entirely through loops, conditions, and console functions.

Key Backend Elements in Your Code:

•**Player Control**: The player's position (playerX) is updated based on keyboard input (A for left, D for right).

•**Alien Movement**: Aliens (alienX, alienY) descend gradually, with their position reset randomly when they reach the bottom.

•**Bullet System**: Bullets (bulletX, bulletY) are fired upward when the spacebar is pressed, and their position is updated each loop until they collide or leave the screen.

•**Collision Detection**: If a bullet hits an alien, the score increases and the alien respawns. If an alien collides with the player, the game ends.

•**Game State Management**: The loop continuously checks for conditions like running, quitting (Q key), or game over.

•**Difficulty Control**: Alien descent speed is controlled by a timer (if (timer++ % 5 == 0)), making gameplay progressively harder.

Core Logic Flow:

•**Input Handling**: Detects player movement and shooting.

•**Game Loop**: Updates positions of player, aliens, and bullets.

•**Collision Checks**: Determines whether bullets hit aliens or aliens hit the player.

•**Score Update**: Adds points when aliens are destroyed.

•**Game Over Condition**: Ends the game when an alien reaches the player's position.

This backend structure ensures smooth gameplay and demonstrates how C++ can be used to manage real-time interactions in a simple console-based environment.

# Chapter 6: Important Features of the Project

**Player Control System**: The player can move left or right using simple keyboard inputs (A and D) and shoot bullets with the spacebar, ensuring intuitive gameplay.

**Alien Respawn Mechanism**: Aliens descend from the top of the screen and respawn at random positions once destroyed or when they reach the bottom, keeping the game dynamic.

**Collision Detection**: The game checks whether bullets hit aliens or aliens collide with the player, updating the score or triggering game-over conditions accordingly.

**Scoring System**: Each alien destroyed adds 10 points to the score, which is displayed in real time at the bottom of the screen.

**Game State Management**: The program handles running, quitting (Q key), and game-over conditions, ensuring smooth control over the session.

**Color-Coded Display**: Different colors are used for the player, aliens, bullets, and ground line, making the console visuals clear and easy to follow.

**Difficulty Progression**: Alien descent speed increases gradually through a timer mechanism, making the game more challenging as time passes.

# Chapter 7: Implementation

**Game Initialization**:
•The game sets up the console window and initializes variables such as player position (playerX), alien position (alienX, alienY), bullet coordinates, score, and game state.
•Randomization (srand(time(NULL))) ensures aliens spawn at different positions each time.

**Game Loop Execution**:
•The main loop runs continuously while running = true.
•The console is refreshed each cycle using SetConsoleCursorPosition, giving the illusion of animation.

**Rendering the Game Area**:
•The player (A), aliens (W), bullets (|), and ground line (-) are drawn on the screen using nested loops.
•Colors are applied with setColor() to distinguish elements.

**User Input Handling**:
•_kbhit() and _getch() detect keyboard input.
•Keys A and D move the player left or right, Space fires a bullet, and Q quits the game.•

**Bullet Movement**:
•When fired, bullets move upward each loop (bulletY--) until they collide with an alien or leave the screen.

**Alien Movement**:
•Aliens descend gradually (alienY++) and respawn at random positions when they reach the bottom.
•A timer (if (timer++ % 5 == 0)) controls descent speed, increasing difficulty over time.

**Collision Detection**:
•If a bullet hits an alien, the score increases by 10, and the alien respawns.
•If an alien collides with the player at the bottom row, the game ends with a “GAME OVER” message.

**Score Display and Game State**:
•The score is updated and displayed after each frame.
•The game loop continues until the player quits or loses.

# Chapter 8: Advantages and Limitations

## 8.1 Advantages

The Alien Invasion game offers several advantages in terms of design, performance, and user experience:

- Lightweight Execution: Since the game runs in the terminal using C++, it requires minimal system resources and works smoothly on standard computers.

- Simple Controls: The use of basic keyboard inputs (A, D, Space, Q) makes the game easy to learn and play.

- Real-Time Interaction: Player movement, alien descent, bullet firing, and collision detection are processed instantly, providing responsive gameplay.

- Color-Coded Display: Different colors for the player, aliens, bullets, and ground line improve visibility and make the console interface engaging.

- Progressive Difficulty: Alien descent speed increases gradually, keeping the game challenging and improving replay value.

- Educational Value: The project demonstrates practical use of loops, conditionals, randomization, and collision detection, making it a strong academic exercise.

## 8.2 Limitations

Despite its effective functionality, the Alien Invasion game has certain limitations identified during development and testing:

- Basic Graphics: The game uses console characters instead of advanced graphics or animations, limiting visual appeal.

- Single-Player Only: No multiplayer or online functionality is included.
- No Sound Effects: The absence of audio feedback reduces immersion.
- Fixed Controls: The control scheme is hardcoded and cannot be customized.
- No Save Feature: Progress cannot be saved mid-game; restarting resets all variables.
- Platform Dependency: The use of <conio.h> and <windows.h> makes the game dependent on Windows systems.

# Chapter 9: Future Enhancements

The current version of the Alien Invasion game provides a functional arcade-style gameplay experience, and several improvements can be implemented in future versions to enhance gameplay, visuals, and scalability:

1. **Sound Integration**: Adding background music and sound effects for shooting, alien destruction, and game-over events would increase immersion.
2. **Multiple Alien Types**: Introducing different alien characters with varying speeds, strengths, or movement patterns could diversify gameplay.
3. **Boss Battles**: Special enemies with higher health or unique attack patterns could appear after certain waves, adding challenge.
4. **Power-Ups**: Features like extra lives, faster bullets, or shields could be introduced to reward skilled players.
5. **Customizable Controls**: Allowing players to remap keys would improve accessibility and user experience.
6. **Difficulty Modes**: Options such as Easy, Medium, and Hard could be added to cater to different skill levels.
7. **Cross-Platform Support**: Adapting the game to run on Linux or macOS by replacing Windows-specific libraries (<conio.h>, <windows.h>) with portable alternatives.
8. **Graphical Upgrade**: Moving from console characters to a graphical interface using libraries like SDL or OpenGL could modernize the game.
9. **Save/Resume Feature**: Implementing a system to save progress and resume later would enhance replayability.

# Chapter 10: Conclusion

The Alien Invasion game developed in C++ is a successful demonstration of how fundamental programming concepts can be applied to create an interactive and entertaining application. By combining loops, conditionals, randomization, collision detection, and console handling, the project delivers a real-time gaming experience within a simple terminal environment. The game highlights the importance of structured logic, user-friendly controls, and responsive feedback, all of which contribute to smooth gameplay.

Beyond its entertainment value, the project serves as an academic exercise that bridges theoretical learning with practical implementation. It shows how even basic console-based programs can incorporate real-time interaction, scoring systems, and progressive difficulty, making it both engaging for players and valuable for students learning C++. While the current version is limited in terms of graphics, sound, and advanced features, it lays a strong foundation for future enhancements such as power-ups, boss battles, and cross-platform support.

In conclusion, the Alien Invasion game is not just a fun project but also a meaningful step in mastering programming skills, problem-solving, and creative application of C++ in game development.

# Chapter 11: References

The following references and resources were consulted during the research, design, and development of this project:

Book References:

1. E. Balagurusamy, Object-Oriented Programming with C++, McGraw Hill.
2. Yashavant Kanetkar, Let Us C, BPB Publications.

Online References:

1. Official C++ documentation: cplusplus.com
2. Microsoft Docs for Windows Console Functions: docs.microsoft.com

Other References:

1. Course Notes: Object Oriented Programming with C++– BCA 2nd Semester, Dev Bhoomi Uttarakhand=University.
2. Lectures and practical demonstrations by Ms.Dhruvi Dagar, Faculty, Department of= Computer Applications.
3. GitHub open-source C++ projects for reference of GUI design patterns.

**— End of Report —**

Alien Invasion Game | Paras Thapa| BCA (AI&DS) 2nd Semester Dev Bhoomi Uttarakhand University, Dehradun | April 2026